

VIXOR

Workflow Agent Report

Full Project Audit and Execution Plan

From current state to production-ready (300+ daily users)

Repository: kam65624-cmd/vixor-APP

Production: vixor-app.vercel.app

Date: 2026-06-30

Classification: Confidential

Table of Contents

1. Executive Summary	3
1.1 Key Metrics	3
2. Chart Analysis Bug Diagnosis	4
2.1 Pipeline Architecture	4
2.2 Root Cause Analysis	4
2.3 Fix Plan	4
3. UI/UX Design System Overhaul	6
3.1 Current Design System Audit	6
3.2 Page-by-Page Audit	6
3.3 New Design System Specification	8
4. Production Hardening	10
4.1 Serverless Function Optimization	10
4.2 Rate Limiting and Abuse Prevention	10
4.3 Task Breakdown	10
5. Technical Debt Cleanup	12
5.1 Type Safety	12
5.2 Dead Code and Dependencies	12
5.3 Task Breakdown	12
6. Execution Timeline	13
6.1 Phase 1 Detail: Chart Analysis Fix (IMMEDIATE)	13
6.2 Phase 2 Detail: Design System Overhaul	13
6.3 Phase 3-4: Production + Tech Debt	13
7. Complete Task Registry	14

1. Executive Summary

This report presents a comprehensive audit of the VIXOR trading platform and a complete execution plan to bring the application from its current state to a production-ready product capable of serving 300+ daily users. The audit was conducted with full file-system access to the entire codebase (363 TypeScript files across 16 domains, 92 runtime libraries), and covers four critical areas: the broken chart analysis pipeline, the UI/UX design system overhaul, production hardening, and remaining technical debt.

The most critical finding is that the chart analysis feature is broken due to the VLM vision step failing silently on Vercel serverless functions. The z-ai-web-dev-sdk VLM call (chart-vision.ts) likely times out or encounters an import issue in the serverless environment, causing the entire analysis pipeline to produce no useful output for users. Additionally, the UI suffers from an inconsistent design system with multiple empty stub pages that degrade the user experience. The current dark theme uses a flat oklch palette that lacks visual hierarchy and fails to create the professional trading terminal aesthetic required for a product used by active traders.

The execution plan is organized into four sequential phases, each with clear tasks, file-level specificity, acceptance criteria, and estimated effort. Phase 1 (Critical Fix) addresses the chart analysis bug immediately. Phase 2 (Design System Overhaul) replaces the current CSS custom properties and redesigns all empty pages. Phase 3 (Production Hardening) prepares the app for 300+ daily users. Phase 4 (Technical Debt) cleans up the remaining issues identified during this audit.

1.1 Key Metrics

Metric	Value	Status
Total TS files	363	Active
Domain modules	16	Active
Runtime libs	92	Active
Route pages	35	Active
Empty stub pages	2	Need replacement
Pages with mock data	~12	Need real data
Chart analysis	BROKEN	Critical fix needed
Design system	Flat/dull	Full overhaul needed
Vercel deployment	GitHub auto-deploy	Working
Supabase RLS	All tables	Configured

2. Chart Analysis Bug Diagnosis

CRITICAL - User-reported: analysis not working

The chart analysis pipeline is the core feature of VIXOR. When a user uploads a chart screenshot, the system should extract the trading pair and price from the image using a VLM (Vision Language Model), then run the local SMC/ICT analysis engine on real OHLCV data to produce a comprehensive technical analysis. The user reports this entire flow is broken: uploads do not produce results, or the analysis hangs indefinitely.

2.1 Pipeline Architecture

The analysis pipeline has five stages: (1) Chart Vision - extract context from the uploaded image using z-ai VLM; (2) Validate - soft validation of the extracted context; (3) Truth Validation - compare vision price against real market data; (4) Local Engine - run SMC/ICT analysis on real OHLCV data; (5) Build Result - compose the final AnalysisResult. The pipeline is designed to be fault-tolerant: if the VLM fails, the engine still runs on the user-selected pair. If real OHLCV data is unavailable, the engine uses synthetic data as a fallback. Theoretically, the pipeline should always produce a result.

2.2 Root Cause Analysis

After auditing all files in the pipeline, the most likely root causes are as follows. The VLM call in chart-vision.ts (line 74) uses zai.chat.completions.createVision() with the glm-4.6v model. This call requires the z-ai-web-dev-sdk to be properly initialized on the server. While the SDK is installed in node_modules, Vercel serverless functions have a limited execution context. The dynamic import (line 33: const ZAI = await import("z-ai-web-dev-sdk")) may fail silently if the SDK binary or native dependencies are not bundled correctly for the serverless environment. The error is caught on line 144 and logged as a warning, but the analysis continues with null chartContext, meaning no symbol is detected from the image.

2.2.1 Suspected Issues (ranked by likelihood)

#	Issue	Severity	File	Evidence
1	VLM SDK timeout on Vercel serverless	CRITICAL	chart-vision.ts:33-86	Dynamic import + VLM call likely exceeds 10s Vercel function timeout
2	VLM model glm-4.6v unavailable or slow	HIGH	chart-vision.ts:76	Model name hardcoded; no fallback model configured
3	createAnalysis server fn timeout	HIGH	functions.ts:28-316	Full pipeline runs in one server function; VLM + OHLCV fetch + engine = easily > 10s
4	TWELVEDATA_API_KEY not set in Vercel env	MEDIUM	price-fetcher.ts:548	TwelveData klines return [] if no API key; engine falls to synthetic data
5	Image too large for base64 round-trip	LOW	functions.ts:37-38	8MB max enforced on client, but large images may cause memory issues server-side

2.3 Fix Plan

The fix requires a multi-pronged approach. First, the VLM vision step must be made non-blocking and moved to a separate serverless function or executed with a longer timeout. Second, the main createAnalysis server function needs its Vercel function timeout increased from the default 10s to at least 30s (configurable in vercel.json). Third, a fallback model should be configured so that if glm-4.6v fails, the system tries a secondary model before falling back to skipping vision entirely. Fourth, the quickAnalyze path (which skips the image entirely) should be tested independently to confirm the local SMC/ICT engine itself works correctly on real OHLCV data.

2.3.1 Task Breakdown

Task ID	Description	Files	Effort
FIX-1	Increase Vercel function timeout to 30s for analysis routes	vercel.json	5 min
FIX-2	Add error boundary logging around VLM import and call	chart-vision.ts	15 min
FIX-3	Add fallback VLM model configuration	chart-vision.ts	20 min
FIX-4	Test quickAnalyze path independently (no image)	functions.ts, engine.ts	15 min
FIX-5	Verify TWELVEDATA_API_KEY is set in Vercel env vars	Vercel Dashboard	5 min
FIX-6	Add end-to-end analysis test with real data	New test file	30 min

3. UI/UX Design System Overhaul

User feedback: current design is unacceptable

The current design system uses a flat oklch color palette defined in styles.css with minimal visual hierarchy. The user describes the app as looking like a "dead ruin" with a "stinking design system." The primary issues are: (1) the dark theme uses flat, low-contrast colors that lack depth and visual interest; (2) multiple pages are empty stubs that show nothing but a generic coming-soon message; (3) the layout is monotonous with no visual variation between pages; (4) the overall aesthetic does not match the professional trading terminal feel that users expect from a platform handling real money.

3.1 Current Design System Audit

The current CSS custom property system in styles.css defines 50+ tokens organized into semantic layers (background, foreground, card, primary, trading colors). The color space is oklch (OKLAB), which is modern but creates very flat, desaturated colors. The primary green (#10B981) is mapped to oklch(0.72 0.17 162) which appears as a medium-brightness emerald with minimal vibrancy. The background colors are extremely dark (oklch luminance 0.16-0.24) with almost no perceptible difference between card, muted, and accent surfaces. This creates a "flat wall" effect where nothing stands out.

3.1.1 Design System Problems Found

Problem	Current State	Impact	Fix
Flat color palette	oklch with L=0.16-0.24 range	No visual depth, pages look dead	Add subtle gradients, increase L range to 0.10-0.30
No accent gradients	Solid color fills everywhere	Boring, monotonous UI	Add gradient backgrounds to key surfaces
Inconsistent spacing	Mixed inline styles + CSS vars	Layout feels uneven	Standardize spacing scale (4/8/12/16/24/32px)
Empty stub pages	2 pages (arbitrage, activity-web3)	Users see dead-end pages	Replace with meaningful content or hide from nav
No loading states	Generic spinner only	Perceived slowness	Add skeleton loaders, progress indicators
No micro-inter actions	No hover/press feedback	App feels static	Add transition animations on cards/buttons
Typography scale	Only 2 weights used in practice	No visual hierarchy	Implement 4-level type scale (Display/H1/H2/Body)
Experience styles unused	bullx/axiom/opensea tokens exist	Dead code, confusion	Remove or integrate into main theme

3.2 Page-by-Page Audit

All 35 route pages were audited for completeness, design quality, and user value. The pages fall into four categories: fully functional (8 pages), functional but with mock/empty data (12 pages), partial stubs (2 pages), and heavily implemented feature pages (13 pages). The following table summarizes each page and the work needed.

Page	Lines	Status	Work Needed
analyze.tsx	484	Functional	Redesign upload UI, add progress feedback
analysis.\$id.tsx	2061	Functional	Redesign result card layout, add print-friendly view
signals.tsx	320	Functional	Add filter/sort, improve card design
charts.tsx	18	Stub	REBUILD: embed TradingView widget with pair selector
journal.tsx	552	Functional	Redesign entry cards, add rich text editor
daily-loop.tsx	1671	Feature	Reduce inline styles, use PageLayout components
copilot.tsx	2054	Feature	Redesign chat UI, add suggested prompts
index.tsx	493	Dashboard	Redesign dashboard widgets, add market overview
discover.tsx	476	Data pages	Improve token cards, add search/filter
perpetuals.tsx	236	Data pages	Redesign position cards
whale.tsx	167	Data pages	Improve trade list, add pair filter
alpha.tsx	279	Data pages	Improve feed cards, add tab filters
predictions.tsx	211	Data pages	Add accuracy trend chart
curves.tsx	189	Data pages	Improve accumulation bars
yield.tsx	199	Data pages	Improve yield cards
communities.tsx	315	Data pages	Improve strategy cards
pnl.tsx	244	Data pages	Add PnL chart visualization
backtest.tsx	619	Feature	Add parameter input form
experiments.tsx	726	Feature	Improve experiment cards
arbitrage.tsx	17	EMPTY STUB	REBUILD or HIDE from navigation
activity-web3.tsx	18	EMPTY STUB	REBUILD or HIDE from navigation
trade-desk.tsx	606	Feature	Improve layout, add quick actions
profile.tsx	431	Feature	Redesign profile header, add stats
settings.tsx	481	Feature	Group settings into sections
premium.tsx	248	Feature	Redesign pricing cards
rewards.tsx	584	Feature	Improve reward cards
referral.tsx	320	Feature	Redesign referral flow
vision.tsx	191	Data pages	Improve analysis list
pulse.tsx	211	Data pages	Improve market pulse widgets

Page	Lines	Status	Work Needed
trackers.tsx	285	Feature	Improve tracker cards
bags.tsx	297	Feature	Improve bag cards
token.\$symbol.tsx	209	Data pages	Improve token detail view
wallet-web3.tsx	257	Feature	Redesign wallet connect flow
notifications.tsx	222	Data pages	Improve notification cards
admin/api-keys.tsx	64	Admin	Redesign admin panel

3.3 New Design System Specification

The new design system will be built on three pillars: (1) a refined color palette with increased contrast and subtle gradients that create depth without sacrificing the dark terminal aesthetic; (2) a consistent spacing and typography scale that creates visual rhythm across all pages; (3) a component library of reusable patterns (stat cards, data rows, empty states, loading skeletons) that ensure every page looks cohesive and professional. The target aesthetic is a modern trading terminal inspired by Bloomberg Terminal, DexScreener, and BullX, but with VIXOR unique identity.

3.3.1 Color Palette Direction

Background range: primary surface at L=0.12 (very dark blue-black), card surface at L=0.16, elevated surface at L=0.20. This creates a clear 3-level depth hierarchy. Accent color: a vibrant emerald green with higher chroma (C=0.20) for primary actions and bullish signals. Trading semantics: green for bullish (slightly warmer), red for bearish (slightly cooler), amber for neutral/wait. Border colors: very subtle (L=0.25, C=0.02) to define structure without visual noise. Text: primary at L=0.95 for maximum readability, secondary at L=0.65 for labels and metadata, tertiary at L=0.45 for disabled states.

3.3.2 Task Breakdown

Task ID	Description	Files	Effort
UI-1	Rewrite CSS custom properties with new palette	styles.css	2 hrs
UI-2	Create reusable gradient utility classes	styles.css	30 min
UI-3	Redesign PageLayout component (header, tabs, spacing)	PageLayout.tsx	3 hrs
UI-4	Redesign EmptyState component (illustrations, CTAs)	EmptyState.tsx	1 hr
UI-5	Create SkeletonLoader component	New component	1 hr
UI-6	Add hover/press micro-interactions to DataRow	PageLayout.tsx	30 min
UI-7	Redesign charts.tsx with TradingView embed	charts.tsx	2 hrs
UI-8	Hide or rebuild arbitrage.tsx stub page	arbitrage.tsx, AppShell.tsx	30 min

Tas k ID	Description	Files	Effort
UI-9	Hide or rebuild activity-web3.tsx stub page	activity-web3.tsx, AppShell.tsx	30 min
UI-10	Redesign analyze.tsx upload experience	analyze.tsx	2 hrs
UI-11	Redesign analysis result page	analysis.Sid.tsx	3 hrs
UI-12	Redesign dashboard (index.tsx) widgets	index.tsx	3 hrs
UI-13	Improve all data page card designs	12 route files	4 hrs
UI-14	Remove unused experience/styles system	experience/styles/	30 min
UI-15	Add responsive desktop layout for lg screens	Multiple files	2 hrs

4. Production Hardening

Target: 300+ daily active users

To support 300+ daily users, the application needs several infrastructure and code-level improvements. The current architecture is sound (TanStack Start + Vercel Serverless + Supabase + Upstash Redis), but there are gaps in rate limiting, error handling, caching strategy, and bundle optimization that must be addressed before scaling. This chapter covers all production readiness tasks organized by priority.

4.1 Serverless Function Optimization

Vercel serverless functions have a default timeout of 10 seconds for Hobby plans and can be configured up to 60 seconds on Pro. The analysis pipeline currently runs everything in a single server function call (VLM extraction + OHLCV fetching + SMC/ICT engine + DB write), which can easily exceed 10 seconds. This must be split into a multi-step approach: (1) immediate DB insert with "processing" status; (2) background processing via Vercel cron or a separate async function; (3) client polls for completion. Additionally, the Binance WebSocket singleton (binance-ws.ts) manages client-side real-time prices for signal monitoring, but the server-side price fetching lacks connection pooling for TwelveData API calls, which could become a bottleneck at scale.

4.2 Rate Limiting and Abuse Prevention

The application has an Upstash Redis rate limiter (redis-rate-limiter.ts) and a circuit breaker (circuit-breaker.ts), but the rate limiter wrapper is not consistently applied to all server functions. The createAnalysis function, in particular, costs 10 points per invocation and should be rate-limited to prevent abuse. The current implementation checks points balance but does not limit request frequency. A comprehensive rate limiting strategy should be applied: 10 requests/minute for analysis, 60 requests/minute for general queries, and 5 requests/minute for write operations.

4.3 Task Breakdown

Task ID	Description	Files	Effort
PROD -1	Increase analysis server function timeout to 30s	vercel.json	10 min
PROD -2	Implement rate limiter wrapper for all server functions	New middleware	2 hrs
PROD -3	Add connection pooling for TwelveData API	price-fetcher.ts	1 hr
PROD -4	Implement analysis queue with polling (split long ops)	functions.ts, analyze.tsx	3 hrs
PROD -5	Add Redis cache warming on deployment	cache.ts	30 min
PROD -6	Optimize client bundle - remove dead imports	Multiple files	2 hrs
PROD -7	Implement unified error handler for all server functions	New middleware	1.5 hrs

Task ID	Description	Files	Effort
PROD-8	Add structured logging (replace console.log)	structured-logger.ts	2 hrs
PROD-9	Fix sol-price API endpoint (currently broken)	AppShell.tsx, route	30 min
PROD-10	Add health check endpoint for monitoring	New route	30 min
PROD-11	Configure Vercel monitoring and alerting	Vercel Dashboard	30 min
PROD-12	Optimize image upload (compression before send)	analyze.tsx	1 hr

5. Technical Debt Cleanup

Lower priority but necessary for long-term maintainability

Beyond the critical fixes and the design overhaul, the codebase has accumulated technical debt that should be addressed to ensure long-term maintainability and developer productivity. This includes type safety issues (9 any types that should be properly typed), dead dependencies that inflate the bundle size, excessive console.log statements that leak information in production, and some PII exposure risks in webhook handlers and copilot stream responses.

5.1 Type Safety

The codebase has at least 9 explicit any type usages that should be replaced with proper TypeScript types. The most concerning are in the analysis functions (functions.ts lines 227-228, 238-241, 249-250, 287-288, 364-365) where Supabase query results are cast to any. These should use the generated Supabase types from src/shared/supabase/types.ts. The route files (whale.tsx, perpetuals.tsx, curves.tsx, predictions.tsx) use any for server response data, which should be typed with proper interfaces.

5.2 Dead Code and Dependencies

The experience/styles/ directory contains three complete design token sets (bullx, axiom, opensea) that are exported but never imported by any route or component. The WorkspaceSwitcher component references them, but the workspace switching feature does not appear to be active in the current UI. These should be removed or integrated. Additionally, the LLM provider system (src/shared/lm/) contains providers for Anthropic, OpenAI, Groq, and z-ai, but the analysis pipeline explicitly does not use any external LLM. These providers are only used by the Copilot and Debate features, which should be verified as active before keeping the code.

5.3 Task Breakdown

Task ID	Description	Files	Effort
DEBT-1	Replace 9 any types with proper TS types	functions.ts, route files	2 hrs
DEBT-2	Remove unused experience/styles system	experience/styles/	30 min
DEBT-3	Remove excessive console.log statements	Multiple files	1 hr
DEBT-4	Fix PII exposure in stars-webhook	webhook handler	30 min
DEBT-5	Fix copilot-stream potential leak	copilot stream handler	30 min
DEBT-6	Audit and clean dead dependencies	package.json	1 hr
DEBT-7	Add JSDoc comments to all domain functions	domains/	3 hrs

6. Execution Timeline

The following timeline shows when each phase begins and ends, with clear milestones. The total estimated effort is approximately 40-50 hours of focused development work, spread across the four phases. Phase 1 is the highest priority and begins immediately. Phase 2 (the UI/UX overhaul that the user specifically asked about) begins as soon as the chart analysis is confirmed working.

Phase	Start	Duration	Key Milestone	Status
Phase 1: Critical Fix	Immediate	2 hours	Chart analysis produces results	PENDING
Phase 2: Design System	After Phase 1	20 hours	All pages use new design system	PENDING
Phase 3: Production	During Phase 2	12 hours	Rate limiting + error handling ready	PENDING
Phase 4: Tech Debt	After Phase 3	8 hours	Zero any types, clean console	PENDING

6.1 Phase 1 Detail: Chart Analysis Fix (IMMEDIATE)

This phase starts NOW. The tasks are ordered by impact and speed. FIX-1 (increase timeout) and FIX-2 (error boundary logging) are the fastest changes that will provide immediate diagnostic value. Once deployed, we can check Vercel function logs to see exactly where the VLM call fails. FIX-4 (test quickAnalyze independently) will confirm whether the local SMC/ICT engine works at all on Vercel. If quickAnalyze works but createAnalysis does not, the issue is confirmed to be in the VLM/image handling path, not in the analysis engine itself.

6.2 Phase 2 Detail: Design System Overhaul

This is the phase the user specifically asked about ("متى نبدأ" = "When do we start?"). It begins immediately after Phase 1 is confirmed complete. The work is divided into three waves: Wave A (UI-1 through UI-6) rebuilds the foundation (CSS tokens, PageLayout, EmptyState, SkeletonLoader, micro-interactions). Wave B (UI-7 through UI-12) redesigns the highest-traffic pages (charts, analyze, analysis result, dashboard). Wave C (UI-13 through UI-15) polishes all remaining data pages and adds responsive desktop layout. Each wave ends with a deploy to production so the user can see progress incrementally.

6.3 Phase 3-4: Production + Tech Debt

Phase 3 and Phase 4 can run in parallel with the later waves of Phase 2. Production hardening tasks (PROD-1 through PROD-12) are mostly independent of UI work and can be done simultaneously. Technical debt cleanup (DEBT-1 through DEBT-7) is the lowest priority but should be completed before the application is marketed to the 300+ user target audience. The combined timeline for Phases 2-4 is approximately 30-40 hours of development, which can be compressed by parallelizing UI work with production and debt tasks.

7. Complete Task Registry

This is the master list of all tasks across all phases. Each task has a unique ID, clear description, specific files to modify, estimated effort, and acceptance criteria. No additional tasks should be needed to bring the project from its current state to production-ready. This registry is the single source of truth for all remaining work.

ID	Task	Files	Effort	Acceptance Criteria
FIX-1	Increase Vercel function timeout to 30s	vercel.json	10 min	Analysis functions do not timeout
FIX-2	Add error boundary logging around VLM import	chart-vision.ts	15 min	VLM errors logged with full stack trace
FIX-3	Add fallback VLM model configuration	chart-vision.ts	20 min	Secondary model tried on primary failure
FIX-4	Test quickAnalyze path independently	functions.ts	15 min	quickAnalyze returns valid result on Vercel
FIX-5	Verify TWELVEDATA_API_KEY in Vercel env	Vercel Dashboard	5 min	Real OHLCV data used for forex pairs
FIX-6	Add end-to-end analysis test	New test file	30 min	Test passes with real data
UI-1	Rewrite CSS custom properties	styles.css	2 hrs	New palette visible on all pages
UI-2	Create gradient utility classes	styles.css	30 min	Gradients render correctly
UI-3	Redesign PageLayout component	PageLayout.tsx	3 hrs	All pages use new layout
UI-4	Redesign EmptyState component	EmptyState.tsx	1 hr	Empty states show helpful content
UI-5	Create SkeletonLoader component	New component	1 hr	Skeleton shown during loading
UI-6	Add micro-interactions to DataRow	PageLayout.tsx	30 min	Hover/press animations work
UI-7	Redesign charts page	charts.tsx	2 hrs	TradingView chart embedded with pair selector
UI-8	Hide arbitrage stub from nav	AppShell.tsx	30 min	Arbitrage not visible in sidebar
UI-9	Hide activity-web3 stub from nav	AppShell.tsx	30 min	Activity not visible in sidebar
UI-10	Redesign analyze upload experience	analyze.tsx	2 hrs	Upload flow is smooth and intuitive
UI-11	Redesign analysis result page	analysis.\$id.tsx	3 hrs	Result cards are readable and shareable

ID	Task	Files	Effort	Acceptance Criteria
UI-12	Redesign dashboard widgets	index.tsx	3 hrs	Dashboard shows actionable data
UI-13	Improve data page card designs	12 route files	4 hrs	All data pages consistent
UI-14	Remove unused experience/styles	experience/styles/	30 min	No dead code in bundle
UI-15	Add responsive desktop layout	Multiple files	2 hrs	Lg screens use 2-3 column grids
PROD-1	Increase analysis timeout	vercel.json	10 min	Done (merged with FIX-1)
PROD-2	Implement rate limiter wrapper	New middleware	2 hrs	All server functions rate-limited
PROD-3	Add TwelveData connection pooling	price-fetcher.ts	1 hr	No connection errors under load
PROD-4	Implement analysis queue with polling	functions.ts	3 hrs	Analysis completes without timeout
PROD-5	Add Redis cache warming	cache.ts	30 min	Cache warmed on cold start
PROD-6	Optimize client bundle	Multiple files	2 hrs	Bundle size reduced by 20%+
PROD-7	Unified error handler	New middleware	1.5 hrs	All errors return structured JSON
PROD-8	Structured logging	structured-logger.ts	2 hrs	No console.log in production
PROD-9	Fix sol-price endpoint	AppShell.tsx	30 min	SOL price shows in header
PROD-10	Health check endpoint	New route	30 min	/api/health returns 200
PROD-11	Configure Vercel monitoring	Vercel Dashboard	30 min	Alerts configured
PROD-12	Optimize image upload	analyze.tsx	1 hr	Images compressed before upload
DEBT-1	Replace 9 any types	functions.ts, routes	2 hrs	Zero any types in codebase
DEBT-2	Remove unused experience/styles	experience/styles/	30 min	No dead design tokens
DEBT-3	Remove excessive console.log	Multiple files	1 hr	Production logs are clean

ID	Task	Files	Effort	Acceptance Criteria
DEB T-4	Fix PII exposure in webhook	webhook handler	30 min	No PII in logs
DEB T-5	Fix copilot-stream leak	copilot stream	30 min	No data leak in stream
DEB T-6	Audit dead dependencies	package.json	1 hr	All deps are used
DEB T-7	Add JSDoc to domain functions	domains/	3 hrs	All public functions documented